
Final Assignment : Schedule Search

ENGUEHARD Mathis

26 Avril 2026

Table des matières

1	Introduction	2
2	Step 1 : Measuring the WCET of τ_1	2
2.1	Protocol	2
2.2	Statistical Results	2
2.3	WCET Retained and Normalisation	2
3	Step 2 : Schedulability Analysis	3
3.1	Task Set Overview	3
3.2	Utilisation Test	3
3.3	Hyperperiod	3
4	Step 3 : Non-preemptive Scheduling	4
4.1	Schedule 1 : Baseline (No missed deadlines)	4
4.2	Schedule 2 : τ_5 allowed to miss a deadline	5
4.3	Performance Analysis	5

1 Introduction

L'objectif de ce projet est de modéliser et de vérifier l'ordonnabilité d'un système temps réel strict composé de sept tâches périodiques. La démarche adoptée se veut rigoureuse et ancrée dans les concepts fondamentaux du cours.

L'analyse est segmentée en trois phases clés :

1. **Mesure du WCET** : Détermination empirique du temps d'exécution maximal de la tâche τ_1 via des mesures sous Linux.
2. **Schedulability Analysis** : Preuve mathématique de la faisabilité du système en utilisant le test d'utilisation.
3. **Non-preemptive EDF Scheduler** : Implémentation d'un ordonnanceur Earliest Deadline First, et étude comparative lorsqu'une tâche (τ_5) est dépriorisée (autorisée à rater sa *deadline*).

Le code `part1.c` a été rédigée en C standard, compilée avec `gcc` tandis que les fichiers `part2.cpp` et `part3.cpp` sont rédigés en c++, et compilés avec `g++`. L'ensemble des fichiers tourne sous un environnement Linux (WSL), garantissant des mesures temporelles fiables.

2 Step 1 : Measuring the WCET of τ_1

2.1 Protocol

La tâche τ_1 effectue la multiplication de deux entiers de 64 bits (générés aléatoirement entre 10^9 et 2×10^9). Nous effectuons cette opération un très grand nombre de fois (10 000 fois) afin d'obtenir une meilleure précision.

Le chronométrage a été réalisé via l'API POSIX `clock_gettime` couplée à l'horloge `CLOCK_MONOTONIC`. Cette horloge matérielle offre une résolution nanoseconde et est insensible aux ajustements NTP, ce qui est crucial pour profiler du code embarqué ou système.

2.2 Statistical Results

Les durées enregistrées ont été ordonnées (via `qsort`) pour en extraire la distribution statistique.

Quantile	Value (ns)
Min (0%)	65
Q1 (25%)	66
Q2 (Median)	67
Q3 (75%)	67
Max (WCET)	11 807

TABLE 1 – Distribution des temps d'exécution sur 10 000 runs

Les quartiles Q1 à Q3 démontrent que l'opération arithmétique prend structurellement ~ 66 ns. La valeur maximale, considérablement plus élevée, illustre les aléas liés au système d'exploitation sous-jacent (OS jitter).

2.3 WCET Retained and Normalisation

Dans un contexte de temps réel strict, nous devons dimensionner le système pour le pire cas. Nous retenons donc $C_1 = 11807$ ns. Cependant, la période de la tâche étant $T_1 = 10$ ms = 10^7 ns, le ratio C_1/T_1 est insignifiant ($< 0,003\%$). Pour s'aligner sur les unités abstraites de l'énoncé, nous posons :

$$C_1 = 1 \text{ unité abstraite}$$

3 Step 2 : Schedulability Analysis

3.1 Task Set Overview

Le système comporte 7 tâches à *deadlines* implicites ($D_i = T_i$).

Task	C_i (Execution)	T_i (Period)	U_i (Utilisation)	Jobs / Hyperperiod
τ_1	1	10	0,1000	8
τ_2	3	10	0,3000	8
τ_3	2	20	0,1000	4
τ_4	2	20	0,1000	4
τ_5	2	40	0,0500	2
τ_6	2	40	0,0500	2
τ_7	3	80	0,0375	1
Total			0,7375	29

TABLE 2 – Propriétés du système de tâches

3.2 Utilisation Test

Pour un système sur un processeur unique, la condition nécessaire de faisabilité requiert que la charge globale (Total Utilisation) n'excède pas la capacité du CPU ($U \leq 1$).

$$U = \sum \frac{C_i}{T_i} = 0,7375 < 1$$

Le test est validé. Le processeur est sollicité à 73,75%, garantissant un *idle time* de 26,25%.

3.3 Hyperperiod

L'analyse dynamique s'effectue sur l'intervalle de répétition fondamental du système, l'hyperpériode H .

$$H = \text{LCM}(10, 10, 20, 20, 40, 40, 80) = 80 \text{ unités}$$

Une simulation couvrant ces 80 unités (soit 29 jobs) est mathématiquement suffisante pour valider le comportement du système à l'infini.

4 Step 3 : Non-preemptive Scheduling

L'ordonnancement est généré via un script en C++ implémentant la politique **EDF (Earliest Deadline First)** en mode **non-préemptif**. La complexité de sélection du *job* à exécuter est de $\mathcal{O}(N)$, où N est le nombre de jobs actifs à l'instant t .

4.1 Schedule 1 : Baseline (No missed deadlines)

Job	Release	Deadline	Start	End	Waiting Time	Status
$\tau_1[0]$	0	10	0	1	0	OK
$\tau_2[0]$	0	10	1	4	1	OK
$\tau_3[0]$	0	20	4	6	4	OK
$\tau_4[0]$	0	20	6	8	6	OK
$\tau_5[0]$	0	40	8	10	8	OK
$\tau_1[1]$	10	20	10	11	0	OK
$\tau_2[1]$	10	20	11	14	1	OK
$\tau_6[0]$	0	40	14	16	14	OK
$\tau_7[0]$	0	80	16	19	16	OK
$\tau_1[2]$	20	30	20	21	0	OK
$\tau_2[2]$	20	30	21	24	1	OK
$\tau_3[1]$	20	40	24	26	4	OK
$\tau_4[1]$	20	40	26	28	6	OK
$\tau_1[3]$	30	40	30	31	0	OK
$\tau_2[3]$	30	40	31	34	1	OK
$\tau_1[4]$	40	50	40	41	0	OK
$\tau_2[4]$	40	50	41	44	1	OK
$\tau_3[2]$	40	60	44	46	4	OK
$\tau_4[2]$	40	60	46	48	6	OK
$\tau_5[1]$	40	80	48	50	8	OK
$\tau_1[5]$	50	60	50	51	0	OK
$\tau_2[5]$	50	60	51	54	1	OK
$\tau_6[1]$	40	80	54	56	14	OK
$\tau_1[6]$	60	70	60	61	0	OK
$\tau_2[6]$	60	70	61	64	1	OK
$\tau_3[3]$	60	80	64	66	4	OK
$\tau_4[3]$	60	80	66	68	6	OK
$\tau_1[7]$	70	80	70	71	0	OK
$\tau_2[7]$	70	80	71	74	1	OK
Total					108	0 missed

TABLE 3 – Trace d'exécution pour l'algorithme EDF non-préemptif

Synthesis : Le système est parfaitement ordonnançable. Le **Total Waiting Time** s'élève à 108 unités. L'**Idle Time** calculé est de 21 unités ($80 - 59$), validant les 26,25% de temps libre prévus.

4.2 Schedule 2 : τ_5 allowed to miss a deadline

Pour étudier l'impact d'une relaxation de contrainte, τ_5 a été convertie en *background task*. Dans notre code C++, elle est ignorée par le tri EDF et n'est planifiée que lorsque la *ready queue* est vide.

Job	Release	Deadline	Start	End	Waiting Time	Status
$\tau_1[0]$	0	10	0	1	0	OK
$\tau_2[0]$	0	10	1	4	1	OK
$\tau_3[0]$	0	20	4	6	4	OK
$\tau_4[0]$	0	20	6	8	6	OK
$\tau_6[0]$	0	40	8	10	8	OK
$\tau_1[1]$	10	20	10	11	0	OK
$\tau_2[1]$	10	20	11	14	1	OK
$\tau_7[0]$	0	80	14	17	14	OK
$\tau_5[0]$	0	40	17	19	17	OK
$\tau_1[2]$	20	30	20	21	0	OK
$\tau_2[2]$	20	30	21	24	1	OK
$\tau_3[1]$	20	40	24	26	4	OK
$\tau_4[1]$	20	40	26	28	6	OK
$\tau_1[3]$	30	40	30	31	0	OK
$\tau_2[3]$	30	40	31	34	1	OK
$\tau_1[4]$	40	50	40	41	0	OK
$\tau_2[4]$	40	50	41	44	1	OK
$\tau_3[2]$	40	60	44	46	4	OK
$\tau_4[2]$	40	60	46	48	6	OK
$\tau_6[1]$	40	80	48	50	8	OK
$\tau_1[5]$	50	60	50	51	0	OK
$\tau_2[5]$	50	60	51	54	1	OK
$\tau_5[1]$	40	80	54	56	14	OK
$\tau_1[6]$	60	70	60	61	0	OK
$\tau_2[6]$	60	70	61	64	1	OK
$\tau_3[3]$	60	80	64	66	4	OK
$\tau_4[3]$	60	80	66	68	6	OK
$\tau_1[7]$	70	80	70	71	0	OK
$\tau_2[7]$	70	80	71	74	1	OK
Total					109	0 missed

TABLE 4 – Trace d'exécution avec τ_5 configurée en *background task*

4.3 Performance Analysis

Metric	Schedule 1 (Standard EDF)	Schedule 2 (τ_5 in background)	Delta
Total Waiting Time	108	109	+1
Execution Time	59	59	0
Idle Time	21	21	0
Missed Deadlines	0	0	0

Conclusion : Contrairement à l'intuition première, assouplir la contrainte sur τ_5 dégrade les performances globales. En EDF natif, τ_5 ($C = 2$) s'intercale rapidement. En la reléguant au second plan, l'ordonnanceur accorde le processeur à τ_7 ($C = 3$). **Bloquer une tâche courte derrière un traitement plus long augmente structurellement le Total Waiting Time du système.** De plus, le *Utilisation Rate* modéré ($\sim 74\%$) garantit que τ_5 ne rate de toute façon

pas sa *deadline* réelle.